



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The State Space is the set of all possible states (positions) that exist in the environment. In our 12×12 grid, the state space contains 144 cells, and each cell like (3, 5) exists only once. It represents the real-world map with a fixed, finite size. The Search Tree is what the algorithm builds internally while searching for a path. It represents all the possible sequences of actions the agent could take from the start position. Unlike the state space, the same state can appear multiple times in the search tree because there are multiple paths to reach it. For example, cell (0, 0) could appear again if the agent goes Up and then comes back Down. This means the search tree can grow much larger than the state space, and without proper handling, it can even become infinite due to loops.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set solves the problem of repeated state exploration and infinite loops. In a grid environment, cycles exist naturally (e.g., going Up then Down brings you back to the same cell). Without the reached set, the algorithm would keep re-expanding already-visited states, wasting resources or looping forever. DFS is the most vulnerable to this problem. Since it always explores the deepest path first, it would follow a cycle endlessly. For example, (0,0) → Up → (0,1) → Down → (0,0) → Up → ... repeating forever, never reaching the goal. The reached set fixes this by recording every explored state. Before expanding a node, the algorithm checks if (nx, ny) not in visited. If the state was already seen, it skips it. This converts Tree Search (can loop infinitely) into Graph Search (each state processed only once), guaranteeing the search terminates.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS is guaranteed to be optimal in our grid because it explores nodes level by level. It first checks all positions 1 step away, then all positions 2 steps away, and so on. Since every move in our grid has an equal cost of 1, the first time BFS reaches the goal, it has found the shortest possible path. It cannot miss a shorter

route because all shorter paths were already explored in earlier levels. DFS produces winding, suboptimal routes because it explores as deep as possible down one branch before backtracking. It picks a direction and commits to it fully, often going far away from the goal before eventually reaching it through a long, indirect path. DFS is only guaranteed to find a path (with a visited set), but not the shortest one, it returns whichever path it stumbles upon first.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

In a 1000×1000 grid, BFS and DFS have very different memory requirements due to how they store nodes during search. BFS has a space complexity of $O(b^d)$, where b is the branching factor and d is the depth of the solution. BFS must store all nodes at the current level in its queue before moving to the next level. In a 1000×1000 grid with a branching factor of 4 (Up, Down, Left, Right), the queue can grow to hold hundreds of thousands of nodes simultaneously. If the food is far away, BFS expands outward in all directions, storing every frontier node in memory. This can easily exhaust available RAM and crash the program. UCS has the same problem since it also keeps all frontier nodes in its priority queue. DFS has a space complexity of only $O(b \times m)$, where m is the maximum depth. DFS only needs to store the nodes along the current path from start to the deepest node, plus the unexplored siblings at each level. In a 1000×1000 grid, the maximum path length is around 2000 steps, so DFS would store roughly $4 \times 2000 = 8000$ nodes at most a tiny fraction of what BFS needs. So on a massive grid, BFS and UCS are likely to run out of memory before finding the goal, while DFS would survive due to its linear memory usage. This is the classic optimality vs. memory trade-off in search algorithms. BFS gives the best path but at a high memory cost, while DFS is memory-efficient but sacrifices path quality.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING